DESIGN AND VERIFICATION OF A 4-BIT REGISTER FILE WITH MULTIPORT READ AND WRITE SUPPORT

Harshit Saini and Rudra Dev Mudgal

Electronics and Communication Engineering (ECE) | VLSI | Digital Design

Introduction

A register file is a fundamental storage element used in digital systems and processor datapaths. It is designed to store a small set of registers and allow fast access through multiple read and write ports. In this project, a 4-bit register file with four registers, one write port, and two independent read ports has been implemented and verified using SystemVerilog.

Problem Statement

The objective of this project is to design a register file consisting of at least four registers, each 4 bits wide, with one write port and two read ports. The design must support simultaneous reading from two different registers while allowing data to be written into a selected register on the rising edge of the clock.

Objectives

- To design a compact 4-bit register file using SystemVerilog.
- To implement one synchronous write port and two independent read ports.
- To verify correct functionality using a self-checking testbench.
- To confirm read and write operations through simulation waveform analysis.

Features

- Four registers, each 4 bits wide.
- One clocked write port.
- Two asynchronous read ports.
- Simple and synthesizable RTL structure.
- Self-checking verification with manual and random test cases.

Design Overview

The design stores data in a memory array declared as mem [0:3], where each location is 4 bits wide. A write operation occurs on the positive edge of the clock when we is asserted, and the selected register address w_addr receives the input data w_data. The read outputs r_data1 and r_data2 continuously reflect the contents of the registers selected by r_addr1 and r_addr2.

RTL CODE

```
`timescale 1ns/1ps
module reg4bit(
input logic clk,
input logic we,
input logic [1:0] w_addr,
input logic [3:0] w_data,
input logic [1:0] r_addr1,
input logic [1:0] r_addr2,
output logic [3:0] r_data1,
output logic [3:0] r_data2
);
reg [3:0] mem [0:3];

always_ff @(posedge clk) begin
if(we)
mem[w_addr] <= w_data;
end
assign r_data1 = mem[r_addr1];
assign r_data2 = mem[r_addr2];

endmodule
```

RTL Explanation

The RTL module has five main parts: clock input, write control, write address and write data inputs, two read addresses, and two read data outputs. The always block uses always_ff @(posedge clk) to ensure synchronous writing only, which is a standard and synthesizable sequential design style in SystemVerilog. The read logic uses assign statements, which makes both read ports combinational and immediately responsive to address changes

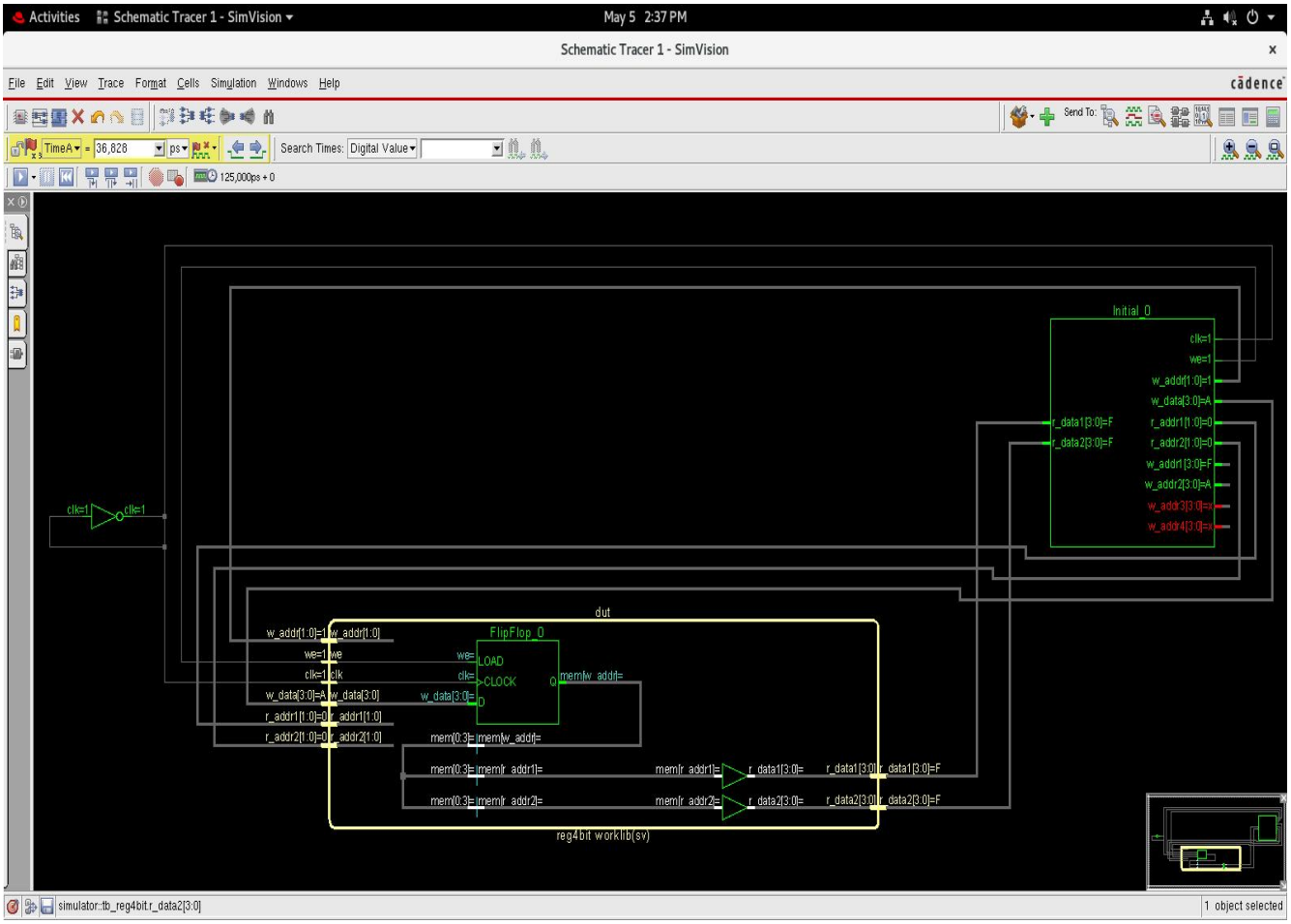
Test Bench

```
`timescale 1ns/1ps
module tb_reg4bit;
logic clk;
logic we;
logic [1:0] w_addr;
logic [3:0] w_data;
logic [1:0] r_addr1;
logic [1:0] r_addr2;
logic [3:0] r_data1;
logic [3:0] r_data2;

reg4bit dut
(.clk(clk),.we(we),.w_addr(w_addr),.w_data(w_data),.r_addr1(r_addr1),.r_addr2(r_addr2),.r_data1(r_da

always #5 clk=~clk;
logic [3:0] w_addr1,w_addr2,w_addr3,w_addr4;
initial begin
clk=0;
we=0;
w_addr = 0;w_data=0;r_addr1 = 0;r_addr2 = 0;
#10;
we=1;
$display("------------------Manual test for write and read---------------");
w_addr = 0;w_data = 4'b1111;
#5; w_addr1 = w_data;
$display(" writing data in 1:%0d",w_data);
#10; w_addr = 1; w_data = 4'b1010;
#5; w_addr2 = w_data;
$display("writing data in 2:%0d",w_data);
#10; we = 0;
#10; r_addr1 = 0; r_addr2 = 1;
#5; $display("Read1 = %b Read2 = %0b",r_data1,r_data2);
if(r_data1 == w_addr1 && r_data2 == w_addr2) begin
$display("--------------------------------SELF CHECKING----------------------------");
$display("test case successfully passsed");
end
else
$display("test case Failed");
#5; $display("---------------manual test finished-----------------");
#10;$display("------------------writing random value and read----------------------");
#2; we=1;
w_addr = 2;w_data = $urandom;w_addr3=w_data;
$display(" Writing Random data in 1:%0d",w_data);
#10; w_addr = 3; w_data = $urandom;w_addr4=w_data;
$display("Writing Random data in 2:%0d",w_data);
#10; we = 0;
#10; r_addr1 = 2; r_addr2 = 3;
#5; $display("Read1 = %b Read2 = %0b",r_data1,r_data2);
if(r_data1 == w_addr3 && r_data2 == w_addr4) begin
$display("Random test case successfully passsed");
end
else
$display("Random test case Failed");
$display("--------------------Random Test Finished----------------------------");
#20; $finish;
end
endmodule
```

Testbench Explaination

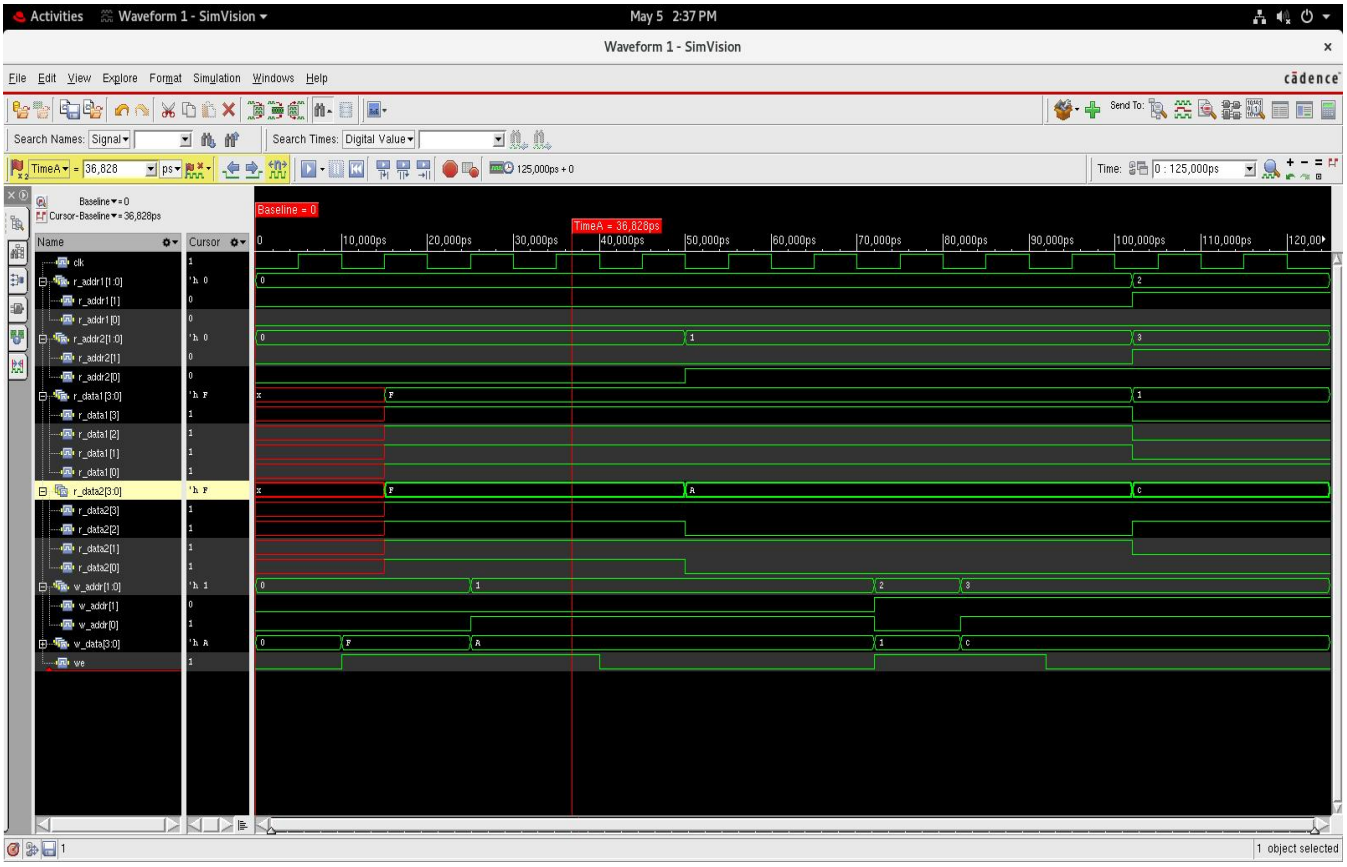
The testbench generates a periodic clock using always #5 clk = ~clk; and applies both directed and random test cases. First, it writes known values like 4'b1111 and 4'b1010 into two registers, then disables write mode and reads them back using the two read ports. After that, it performs random write operations using $urandom and checks whether the read values match the expected stored data. The testbench is self-checking because it prints pass or fail messages based on comparison of expected and actual results.

Block Diagram



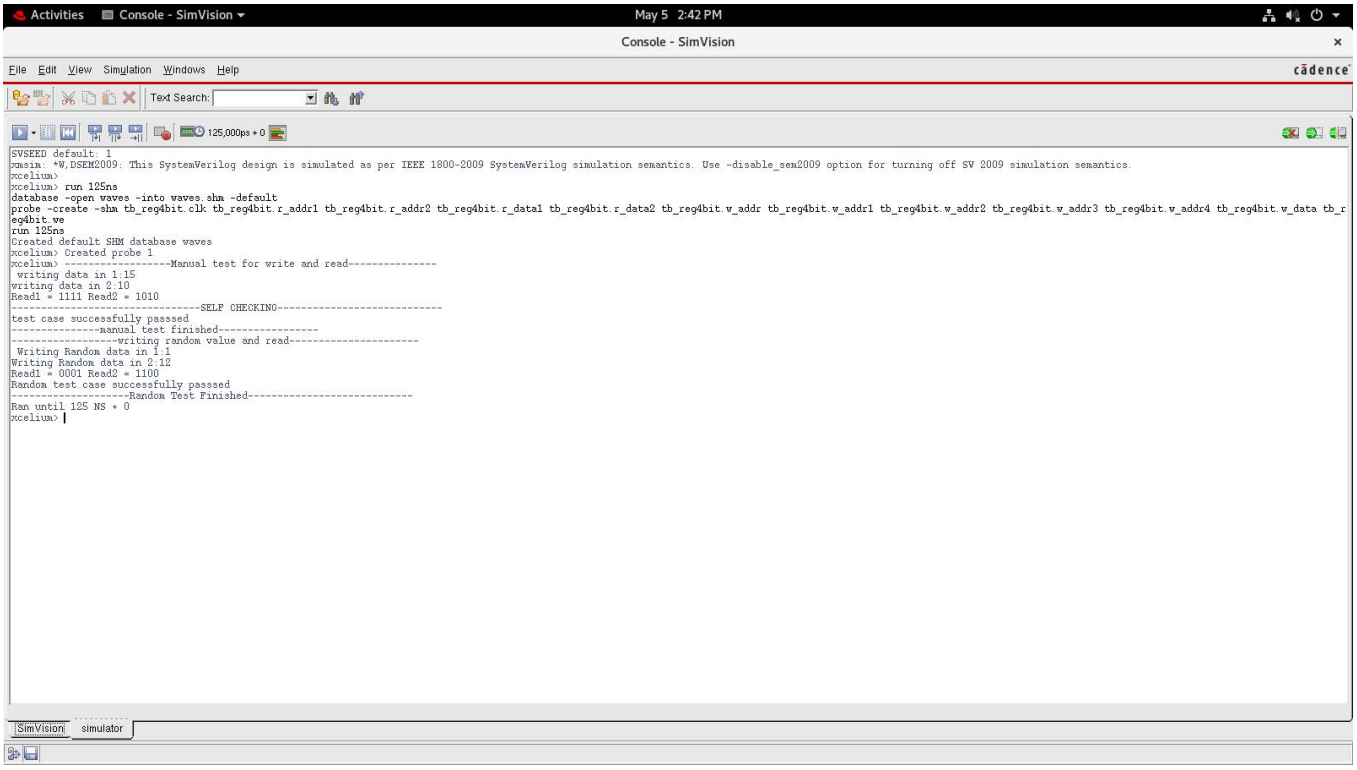
The block diagram of the design contains a clock input, a write enable control, a 4-bit write data bus, two 2-bit read addresses, and two 4-bit read data outputs. Internally, the register file contains a 4x4-bit memory array. The write address selects one register for update, while the two read addresses independently select two registers for output. This structure clearly shows multiport access capability.

Waveform Analysis



The waveform confirms the correct operation of the register file. During the active write phase, the selected memory location updates with the input data at the rising edge of the clock. After write enable is deasserted, both read ports successfully return the stored values from different register addresses. The waveform also shows that random test data is written and read back correctly, proving that the design works for both fixed and random inputs.

Results



The simulation output confirms that the 4-bit register file operates correctly. The manual test case successfully wrote and read back the stored values, and the random test case also passed. The console output clearly shows that the expected read data matched the written values, proving correct functionality of the design.

Future Scope

- The design can be extended to 8-bit or 32-bit register width.
- More registers can be added for larger storage capacity.
- Additional write or read ports can be included for advanced datapath use.
- Reset, enable, and error-checking features can be added for stronger control.

Conclusion

This project successfully implements a simple and synthesizable 4-bit register file with one write port and two read ports using SystemVerilog. The design was verified using a self-checking testbench and simulation waveforms, which confirmed correct write and read behavior. The project provides a strong foundation for understanding register-based storage and multiport memory structures used in digital systems.